CSS

Лекция 1. Основы

О чём лекция

Как писать правила, подключать стили и понимать, какое из них победит

План

1. Зачем нужен CSS
2. Правило: селектор, свойство, значение
3. Три способа подключить стили
4. Наследование
5. Каскад
6. Блочная модель
7. Единицы измерения
8. DevTools

HTML и CSS

	Язык	Отвечает на вопрос
Структура	**HTML**	Что это?
Оформление	**CSS**	Как выглядит?

Определение

CSS — Cascading Style Sheets, каскадные таблицы стилей. Стандарт W3C для описания внешнего вида HTML-документов.

Зачем разделять структуру и оформление

✓ Меньше работы

Правим один файл — меняется весь сайт

✓ Доступность

Разметка по смыслу понятна программам чтения с экрана и поисковикам

✓ Богатый дизайн

Сетки, шрифты, анимации, адаптация под разные экраны

CSS Zen Garden

Один и тот же HTML-документ — десятки разных дизайнов.

Все отличия — **только в стилях**. Ни одного тега `<img>`: картинки заданы фоном.

🔗 csszengarden.com

1. Правило

Пример

```
h1 {
  color: green;
}

p {
  font-size: large;
  font-family: sans-serif;
}
```

- Все `h1` — зелёные
- Все абзацы — крупные, шрифт без засечек

Анатомия правила

```
селектор {
  свойство: значение;
}
```

Часть	Что это
Селектор	К чему применяем
Свойство	Что меняем
Значение	На что меняем
Объявление	Пара `свойство: значение;`

Схема правила

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.

Allow

```
flowchart LR
  A["h1<br/>селектор"] --> B["{ color: green; }<br/>блок объявлений"]
  B --> C["color<br/>свойство"]
  B --> D["green<br/>значение"]
```

Не забывай

- Двоеточие между свойством и значением
- **Точка с запятой** после каждого объявления
- Фигурные скобки `{ }` вокруг блока

⚠ Частая ошибка

Пропущена `;` — это объявление и следующее за ним **не сработают**

Комментарии

```
/* Это комментарий */
body {
  font-size: small;
  /* потом поменяю */
}
```

Браузер их пропускает. Удобно подписывать разделы и временно отключать строки.

Селекторы

По тегу — все элементы этого типа

```
p { color: gray; }
```

Группа — через запятую

```
h1, h2, p { color: gray; }
```

По id — один конкретный элемент

```
#intro { color: purple; }
```

```
<p id="intro">...</p>
```

Практика

Добавьте в `<style>`:

```
h2 { color: maroon; }

p { margin-left: 100px; }

h1 {
  border-bottom: 1px solid maroon;
  padding: 12px;
}
```

Цикл работы

Сохранить → обновить страницу → посмотреть результат

2. Подключение стилей

Три способа

1. **Внешний файл** `.css`
2. **Встроенный** — элемент `<style>`
3. **Inline** — атрибут `style`

1. Внешний файл

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

```
/* styles.css */
h1 { color: green; }
```

✓ Основной способ

Один файл подключается ко **всем** страницам сайта

2. Встроенный

```
<head>
  <style>
    h1 { color: green; }
  </style>
</head>
```

Действует только на **одну страницу**. Подходит для учебных примеров.

3. Inline

```
<h1 style="color: red;">Заголовок</h1>
```

Действует только на **один элемент**.

> **Используем редко**
>
> Смешивает оформление со структурой и неудобен в правке

Сравнение

Способ	Влияет на	Когда
Внешний	Весь сайт	**Всегда**
`<style>`	Одну страницу	Примеры
Inline	Один элемент	Редко

3. Наследование

Документ — это дерево

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.

Allow

```
graph TD
  html --> head
  html --> body
  head --> title
  head --> style
  body --> h1
  body --> p
  body --> h2
  p --> em
```

Семейные связи

Термин	Значение
Родитель	Содержит элемент непосредственно
Ребёнок	Вложен непосредственно
Потомок	Любой вложенный элемент
Предок	Любой элемент выше по дереву
Сёстры	Элементы с одним родителем

`p` — родитель для `em`, `body` — предок для `em`

Дети наследуют от родителей

```
p {
  font-size: large;
  font-family: sans-serif;
}
```

Тег `<em>` внутри абзаца тоже стал крупным и без засечек, хотя для него правила нет.

Что наследуется

✓ Наследуется (текст)

- цвет текста
- шрифт и его размер
- межстрочный интервал

✕ Не наследуется («коробка»)

- рамки
- отступы
- фон

Используем наследование

```
body { color: blue; }
```

Весь текст внутри `body` станет синим.

Собственное правило сильнее унаследованного:

```
em { color: orange; }
```

4. Каскад

Конфликты стилей

На один элемент могут действовать **несколько правил**, и они могут противоречить друг другу. Кто победит?

1. **Источник** стиля
2. **Специфичность** селектора
3. **Порядок** в файле

1. Источник

От слабого к сильному:

1. Стили браузера (по умолчанию)
2. Стили пользователя
3. **Стили автора** (ваши)
4. Правила с `!important`

!important

```
p { color: blue !important; }
```

Перебить нельзя ничем, кроме другого `!important`.

⚠ Не используем без веской причины

2. Специфичность

```
p      { color: green; }
#intro { color: purple; }
```

```
<p id="intro">Первый</p>
<p>Второй</p>
```

Первый абзац — **фиолетовый**: селектор по `id` точнее, чем по тегу.

Шкала специфичности

От слабого к сильному

тег < класс < `#id` < inline-стиль

Чем точнее выбран элемент — тем больше «вес» правила.

3. Порядок: побеждает последнее

```
p { color: red; }
p { color: blue; }
p { color: green; }
```

Цвет абзацев — **зелёный**

То же внутри одного правила:

```
p {
  color: red;
  color: green;
}
```

Как решается спор

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.

Allow

```
flowchart TD
  A["Два правила спорят"] --> B{"Разные источники?"}
  B -- да --> B1["Побеждает более приоритетный"]
  B -- нет --> C{"Разная специфичность?"}
  C -- да --> C1["Побеждает более точный селектор"]
  C -- нет --> D["Побеждает написанное позже"]
```

Пример

```
<style>
  @import url(external.css); /* h1 { color: red } */
  h1 { color: purple; }
</style>

<h1 style="color: blue">Заголовок 1</h1>
<h1>Заголовок 2</h1>
```

Какого цвета заголовки?

Ответ

✓ Заголовок 1 — синий

Inline-стиль сильнее всех

✓ Заголовок 2 — фиолетовый

Правило в `<style>` стоит позже красного из внешнего файла

5. Блочная модель

Каждый элемент — коробка

Браузер видит любой элемент как прямоугольник.

```
h1, h2, p, em, img {
  border: 1px solid blue;
}
```

- Блочные элементы (`h1`, `p`) занимают **всю ширину**
- Строчные (`em`, `img`) — **только по содержимому**

Из чего состоит коробка

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.

Allow

```mermaid
flowchart TB
  subgraph M["margin — внешний отступ"]
    subgraph B["border — рамка"]
      subgraph P["padding — внутренний отступ"]
        C["content — содержимое"]
      end
    end
  end
```

Четыре слоя

Слой	Что это
`content`	Текст или картинка
`padding`	Воздух между содержимым и рамкой
`border`	Рамка
`margin`	Расстояние до соседей

6. Единицы измерения

Абсолютные

`px` — пиксель

- Привычный, но «жёсткий»
- Подходит для рамок и картинок с фиксированным размером

Для печати

`pt`, `cm`, `mm`, `in` на экране не нужны

Относительные

Единица	От чего считается
`rem`	Размер шрифта `html` (обычно 16px)
`em`	Размер шрифта **этого** элемента
`%`	От другого значения (часто от родителя)
`vw`	1/100 ширины окна
`vh`	1/100 высоты окна

rem

База — 16px

Запись	Результат
`1rem`	16px
`2.5rem`	40px
`10rem`	160px

Пользователь увеличил шрифт в браузере — размеры в `rem` вырастут вместе с ним.

Что выбрать

Что задаём	Единица
Шрифты, отступы, ширины	`rem`, `%`
Рамки, мелкие детали	`px`
Часть экрана	`vw`, `vh`

```
header {
  width: 100vw;
  height: 50vh;
}
```

7. DevTools

Инструменты разработчика

Как открыть

F12 или правая кнопка мыши → **Просмотреть код**

- **Elements** — HTML-дерево страницы
- **Styles** — все правила элемента, перечёркнутые — проигравшие
- **Схема блочной модели**
- Значения можно менять прямо в панели

Изменения не сохраняются в файл — это только предпросмотр

Итоги

1. HTML — структура, CSS — оформление
2. Правило: `селектор { свойство: значение; }`
3. Подключаем через внешний `.css`-файл
4. Текстовые свойства **наследуются**, рамки и фон — нет
5. Каскад: **источник → специфичность → порядок**
6. Элемент — коробка: content, padding, border, margin
7. Размеры: `rem` и `%` для гибкости, `px` для мелочей
8. Не работает стиль — открой **DevTools**

Проверь себя

1. Назовите части правила:

```
blockquote { line-height: 1.5; }
```

2. Какого цвета будут абзацы?

```
p { color: purple; }
p { color: green; }
p { color: gray; }
```

Проверь себя

3. Перепишите короче:

```
p { color: white; }
blockquote { color: white; }
li { color: white; }
```

4. Что не так?

```
blockquote {
  font-size: 1em
  line-height: 150%
  color: gray
}
```

Ответы

✓ 1

Селектор `blockquote`, свойство `line-height`, значение `1.5`

✓ 2

Серый — побеждает последнее правило

✓ 3

```
p, blockquote, li { color: white; }
```

✓ 4

Нет точек с запятой после объявлений

Дальше

Лекция 2: текст, цвет и фон